CSE-1003 • MODULE 3 • CLASS 1

Build with blocks

Riya's bill program has grown. Today we break it into small named blocks, called functions, that each do one job.

TODAY'S SYLLABUS TOPICS

Four topics, one tidy program

1

Functions

What a function is and why we use it

2

Definition: writing functions

Telling the function what it needs and gives back, and writing its steps

3

Call and return

How a call works and what return does

4

Declaration

The menu board that announces a function to the compiler

We write functions together first. The declaration comes once you are comfortable. Next class: call by value and reference, the stack, scope and storage classes. Then recursion.

BEFORE WE START • A REAL-LIFE FUNCTION

One name instead of sixty email IDs

Without a group: type every ID, every time

To: name01@example.edu, name02@example.edu, name03@example.edu, name04@example.edu, name05@example.edu, … 55 more IDs

✗ Slow: 60 IDs to type or paste

✗ One typo and a student misses the mail

✗ A new student joins? Fix every email

With a group: type one name

To: cse-sec2-group

The group remembers all 60 members

Riya | Arjun | Meera | Kabir | Isha | Dev | Anaya | + 53 more

✓ Type one name

✓ Nobody is missed

✓ Add a student once

A function works the same way Give the work a name. Use the name. Change it in one place.

BEFORE WE START • A REAL-LIFE FUNCTION

A function is a name for a block of work

In real life: the class email group



The name
`cse-sec2-group`



The members inside
60 students



Send a mail
`To: cse-sec2-group`



A new student joins
Add them once

The same idea as a function (pseudo-code)



The name
`notify_class`



```
void notify_class(void) {
    send(name01); send(name02); …
}
```



Call it
`notify_class();`



Change the function
Edit one line

Write once | **Use by name** | **Change once**

BEFORE WE START • A REAL-LIFE FUNCTION

Everyday functions: something in, something out



Speed dial

Mom → Call → Phone rings



Washing machine

Dirty clothes → Quick wash → Clean clothes



Maps

Your location → Navigate home → The route

You never see the steps inside. You only need the name, what goes in and what comes out.

BEFORE WE START

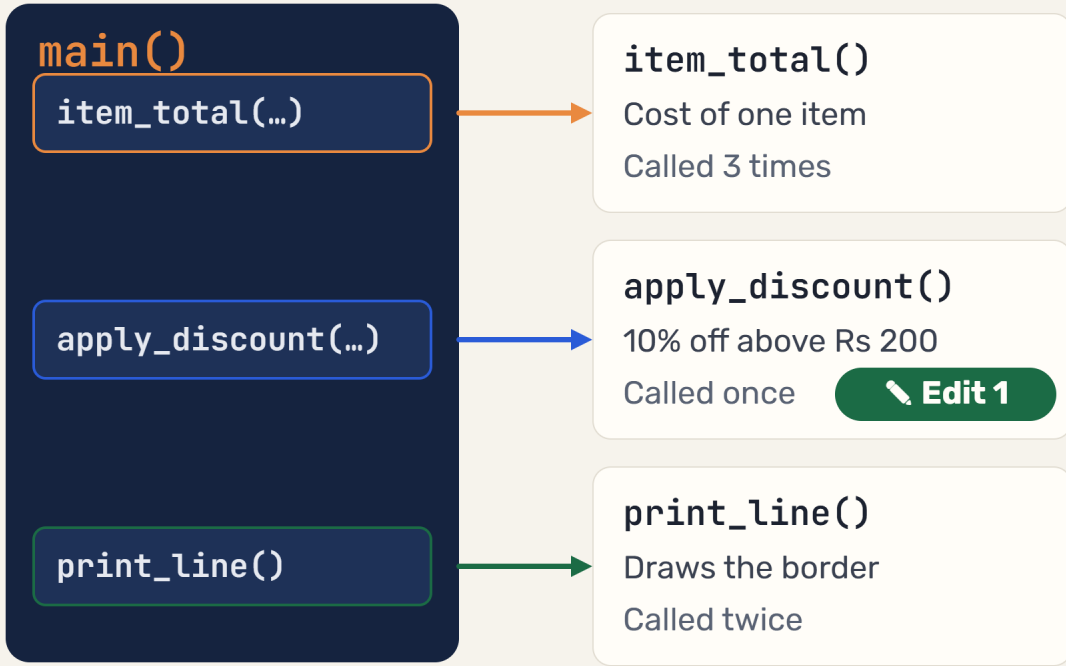
Change the discount: how many edits?

Everything inside main

```
Chai: qty × 10
Discount: if total > 200 … 10%        ✎ Edit 1
Draw border ----------
Samosa: qty × 15
Discount: if total > 200 … 10%        ✎ Edit 2
Draw border ----------
Maggi: qty × 40
Discount: if total > 200 … 10%        ✎ Edit 3
```

The same discount lines are copied again and again.

Split into functions



main reads like a story. Each job lives in one place.

The canteen changes the discount from 10% to 15%. Count your edits:

TODAY'S GOAL

Riya's bill, built from three small functions

item_total works out each item, **apply_discount** decides the price, **print_line** draws the border.

```
How many chai, samosa, Maggi? 10 10 0
---------------------------
Total: Rs 250
To pay: Rs 225
---------------------------
```

TOPIC 1 OF 4 • FUNCTIONS

Now in C: Riya's order becomes a function



You order

“2 chai, please”

`inputs: qty = 2, price = 10`



The chef works

2 × 10

`the body: qty * price`



You get it back

Rs 20

`return value: 20`

```
int item_total(int qty, int price)
```

Riya's chef, written as the first line of a C function.

You have used functions since day one: **printf**, **scanf**, and even **main**.

TOPIC 1 OF 4 • FUNCTIONS

Why functions? Modular programming



Reuse

Write once, use many times: item_total for chai, samosa and Maggi.



Readable

main reads like a story of what happens, not how.



Fix once

A bug in the discount? Fix it in one place.



Teamwork

Each friend writes one function, then you join them.

Like LEGO: small blocks, each with one shape, snap together into something big. That's **modular programming**.

TOPIC 1 OF 4 • FUNCTIONS

Two kinds: ready-made and your own

Library functions

Already written. Include the header and use them.

Function	Header	Job
printf, scanf	stdio.h	Print and read
sqrt, pow	math.h	Square root, power
strlen	string.h	Length of a string (Module 4)

User-defined functions

You write them for your own program.

Function	Who	Job
item_total	you write it	Cost of one item
apply_discount	you write it	10% off above Rs 200
print_line	you write it	Draw a border

TOPIC 1 OF 4 • FUNCTIONS

The function recipe: 5 steps for every function

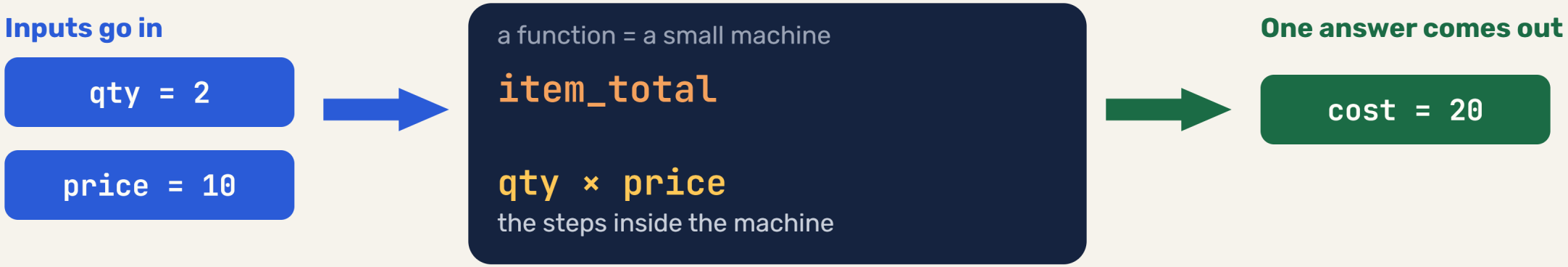
1	**Name the job**	`item_total`
2	**Inputs**	`int qty, int price`
3	**Output**	`int (the cost)`
4	**Write the body**	`return qty * price;`
5	**Test it**	`item_total(2, 10) → 20`

\+ Later today: add the **prototype**, the menu board that announces the function.

First we write functions together. You tell me the logic; I type it.

TOPIC 2 OF 4 · DEFINITION: WRITING FUNCTIONS

Every function answers four questions




```
3      1                2
int item_total(int qty, int price) {
    return qty * price;  4
}
```

1	**What is the job?**	→ the name
2	**What does it need?**	→ the inputs
3	**What does it give back?**	→ the output type
4	**What are the steps?**	→ the body

The idea The code is just the answers to the four questions, written in this order.

Use case 1: Riya's item cost



Riya orders chai at Rs 10 a cup. The canteen needs the cost of any item: quantity times price.

```
item_total(2, 10) → 20
item_total(1, 15) → 15
```

1 • What is the job? Give it a name.	**item_total**
2 • What does it need?	**Two whole numbers: qty and price**
3 • What does it give back?	**One whole number: the cost (int)**
4 • What are the steps?	**Multiply qty by price**

Write it together: item_total

```
int item_total(int qty, int price) {
    int cost = qty * price;
    return cost;
}
```

- Header: output type, name, inputs
- Step: work out the cost
- Give the answer back
- Close the body

item_total(2, 10)	item_total(1, 15)	item_total(0, 40)
20	15	0

Same function, three different calls: each gets its own box with its own qty and price.

Use case 2: the auto fare



An auto charges Rs 30 for the first 2 km and Rs 15 for every km after that. Give the fare for any distance.

```
1 km → 30
2 km → 30
10 km → 150
```

1 • What is the job? Give it a name.

auto_fare

2 • What does it need?

One whole number: km

3 • What does it give back?

One whole number: the fare (int)

4 • What are the steps?

If km is 2 or less: Rs 30.
Otherwise: 30 plus 15 for each km after 2

Write it together: auto_fare

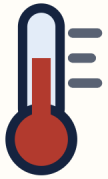
```
int auto_fare(int km) {
    if (km ⩽ 2)
        return 30;
    return 30 + (km - 2) * 15;
}
```

- Header: int out, int in
- Short ride?
- Flat fare
- Rs 30 plus Rs 15 for each extra km
- Close the body

auto_fare(1)	auto_fare(2)	auto_fare(10)
30	30	150

Notice: the first return ends the function at once, so the second line only runs for long rides.

Use case 3: Celsius to Fahrenheit



A weather app gets the temperature in Celsius. Convert it: multiply by 9, divide by 5, then add 32.

```
100 → 212
0 → 32
37 → 98.6 (body temperature)
```

1 • What is the job? Give it a name.

to_fahrenheit

2 • What does it need?

One decimal number: c, the Celsius value

3 • What does it give back?

One decimal number: the Fahrenheit value (float)

4 • What are the steps?

c times 9, divided by 5, plus 32

Write it together: to_fahrenheit

```
float to_fahrenheit(float c) {
    return c * 9 / 5 + 32;
}
```

Header: float out, float in

All the steps in one line

Close the body

`to_fahrenheit(100)`
212.0

`to_fahrenheit(0)`
32.0

`to_fahrenheit(37)`
98.6

Careful! Writing 9 / 5 * c gives 132.0 for 100 degrees, not 212. Why? (Hint: Module 2, integer division.)

TOPIC 2 OF 4 • DEFINITION

Definition: the full recipe



The **recipe card** in the kitchen: every step to make the item.

```
int item_total(int qty, int price)
{
    int t = qty * price;
    return t;
}
```

`int item_total(int qty, int price)`
Header: same as the prototype, without ;

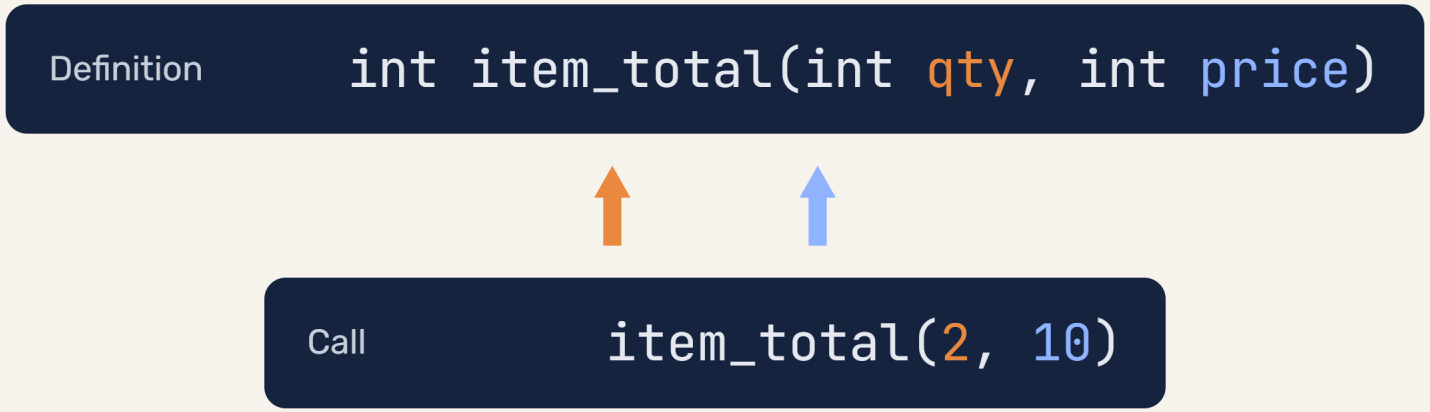
`{ … }`
Body: the steps, between braces

`return t;`
Sends the answer back to the caller

This recipe card is the definition. Later we add the menu board (the declaration) so the compiler knows the function exists.

TOPIC 2 OF 4 • DEFINITION

Parameters and arguments: placeholders and real values



Parameters (in the definition)

Placeholders, like a recipe saying “qty cups of milk”.

Arguments (in the call)

The real values your order fills in: 2 and 10, matched **in order**.

TOPIC 2 OF 4 • DEFINITION

Four shapes a function can have

1 • Nothing in, nothing out




```
void print_line(void)
```

Like a bell: press it and it rings. It needs nothing and gives nothing back.

2 • Something in, nothing out




```
void print_bill(int total)
```

Like a printer: you give it a number, it prints a slip. Nothing comes back.

3 • Nothing in, something out




```
int read_qty(void)
```

Like a ticket machine: press the button and a ticket comes out.

4 • Something in, something out




```
int item_total(int qty, int price)
```

Like the chef: you give an order, you get a cup back.

`void` means **nothing**. In the brackets: nothing goes in. Before the name: nothing comes back.

TOPIC 3 OF 4 • CALL AND RETURN

Call and return: main pauses, the function works, main continues





At the counter you **call** the chef and **wait**. The chef cooks, **returns** the chai, and you carry on from where you were.

Watch a function call in memory

```
1  #include <stdio.h>
2
3  int item_total(int qty, int price) {
4      int t = qty * price;
5      return t;
6  }
7
8  int main(void) {
9      int chai = 2, cost;
10     cost = item_total(chai, 10);
11     printf("Cost: Rs %d\n", cost);
12     return 0;
13 }
```

Memory: the call stack

item_total()

qty	2	@2000
price	10	@2004
t	20	@2008

▲ return 20 to main

main()

chai	2	@1000
cost	20	@1004

Output

```
Cost: Rs 20
```

The line being run

A box that just changed

Each box = one function's memory

@2000 is not @1000: the function's variables are in

6 printf shows the value of cost on the screen.

TOPIC 3 OF 4 • CALL AND RETURN

Four rules of return

1. Sends back ONE value

```
return qty * price;
```

The answer goes back to the caller.

2. Ends the function at once

```
return t;
printf("Done"); /* never runs */
```

Lines after return are skipped.

3. Type must match

```
int item_total(…) → return 20;
```

An int function returns an int.

4. Use the answer!

```
item_total(1, 15); /* lost */
total = item_total(1, 15); /* kept */
```

Store it or print it, or it is thrown away.

One function, called three times

```
1  int item_total(int qty, int price) {
2      return qty * price;
3  }
4
5  int main(void) {
6      int a, b, c, total;
7      a = item_total(2, 10);
8      b = item_total(1, 15);
9      c = item_total(1, 40);
10     total = a + b + c;
11     printf("Total: Rs %d\n", total);
12     return 0;
13 }
```

Memory: the call stack

`item_total()`

qty	1	@2000
price	40	@2004

▲ returned 40

`main()`

a	20	@1000
b	15	@1004
c	40	@1008
total	75	@1012

Output

```
Total: Rs 75
```

The line being run

A box that just changed

Each box = one function's memory

The function is written once. Every call gets its own fresh box.

9 printf shows the total on the screen.

TOPIC 3 OF 4 • CALL AND RETURN • PREDICT

What does it print?

1

```
int sq(int n) { return n * n; }
printf("%d", sq(4) + 1);
```

17

2

```
void hi(void) { printf("Hi "); }
main: hi(); hi(); hi();
```

Hi Hi Hi

3

```
int f(int x) { return x * 2; printf("Done"); }
printf("%d", f(5));
```

10: Done never prints

4

```
int add1(int x) { x = x + 1; return x; }
int a = 5; add1(a); printf("%d", a);
```

5! a did not change. Why? Next class.

CLASS ACTIVITY • TEAM OF THREE • ROUND 1

Three students, three functions, one bill

A — Student A

`item_total`

Job: the cost of one item

Inputs: two whole numbers: qty and price

Gives back: an int: qty × price

Test: item_total(2, 10) → 20

B — Student B

`apply_discount`

Job: 10% off when the total is more than Rs 200

Inputs: one whole number: total

Gives back: an int: the amount to pay

Test: 250 → 225 and 150 → 150

C — Student C

`print_line`

Job: print a border of 28 dashes

Inputs: nothing

Gives back: nothing (void)

Test: prints ----------

1 Pick your card

2 Write your function with the 5-step recipe

3 Show it to the class

4 main joins all three

The rule Each student knows only their own card. The functions never see each other: main joins them.

CLASS ACTIVITY • TEAM OF THREE • REVEAL

Round 1 solution: three functions, then main joins them

Student A

```
int item_total(int qty, int price) {
    return qty * price;
}
```

Student B

```
int apply_discount(int total) {
    if (total > 200)
        return total - total / 10;
    return total;
}
```

Run it with input 10 10 0

```
---------------------------
To pay: Rs 225
---------------------------
```

Student C

```
void print_line(void) {
    int i;
    for (i = 0; i < 28; i++)
        printf("-");
    printf("\n");
}
```

main joins

```
int main(void) {
    int chai, samosa, maggi, total, pay;
    scanf("%d %d %d", &chai, &samosa, &maggi);
    total = item_total(chai, 10)
          + item_total(samosa, 15)
          + item_total(maggi, 40);
    pay = apply_discount(total);
    print_line();
    printf("To pay: Rs %d\n", pay);
    print_line();
    return 0;
}
```

CLASS ACTIVITY · ROUND 2

One more use case: a report card

A — Student A
`total_marks`

Job: add the marks of three subjects

Inputs: three whole numbers: m1, m2, m3

Gives back: an int: the total

Test: total_marks(85, 90, 70) → 245

B — Student B
`average`

Job: the average of three subjects, with decimals

Inputs: one whole number: the total

Gives back: a float: total ÷ 3

Test: average(245) → 81.67 (cast first!)

C — Student C
`grade`

Job: the letter grade: A 90+, B 75+, C 50+, else F

Inputs: one decimal number: avg

Gives back: a char: A, B, C or F

Test: grade(81.67) → B

1. Pick your card
2. Write your function with the 5-step recipe
3. Show it to the class
4. main joins all three

The rule Each student knows only their own card. The functions never see each other: main joins them.

CLASS ACTIVITY · TEAM OF THREE · REVEAL

Round 2 solution: report card

Student A

```
int total_marks(int m1, int m2, int m3) {
    return m1 + m2 + m3;
}
```

Student B

```
float average(int total) {
    return (float) total / 3;
}
```

Student C

```
char grade(float avg) {
    if (avg ⩾ 90) return 'A';
    if (avg ⩾ 75) return 'B';
    if (avg ⩾ 50) return 'C';
    return 'F';
}
```

main joins

```
int main(void) {
    int m1, m2, m3, total;
    float avg;
    scanf("%d %d %d", &m1, &m2, &m3);
    total = total_marks(m1, m2, m3);
    avg = average(total);
    printf("Total %d, Average %.2f\n", total, avg);
    printf("Grade %c\n", grade(avg));
    return 0;
}
```

Run it with input 85 90 70

```
Total 245, Average 81.67
Grade B
```

YOUR TURN · PARSONS PROBLEM

Put the lines in order to build apply_discount

```
A   return total;
B   }
C   int apply_discount(int total) {
D   return total - total / 10;
E   if (total > 200)
```

The function should give 10% off when the total is **more than 200**, and the full total otherwise.

Write the order of letters. Hint: start with the header.

C · E · D · A · B

Header first (C), then the check (E) and its return (D). Line A runs only when the check is false, and } (B) closes the body. Test: 250 → 225, 150 → 150.

TOPIC 3 OF 4 • CALL AND RETURN • MISSING LINE CHALLENGE

One line is missing. Which line, and where?

```c
/* One line is missing. Find it and write it. */
#include <stdio.h>

int apply_discount(int total) {
    int pay = total;
    if (total > 200)
        pay = total - total / 10;
    /* ??? missing line ??? */
}

int main(void) {
    printf("To pay: Rs %d\n", apply_discount(250));
    return 0;
}
```

The compiler warns: **“control reaches end of non-void function”**, and the program prints a **garbage** number. What is missing?

Correct output

```
To pay: Rs 225
```

Answer: return pay; on line 8. An int function must send an answer back; without return, main gets a garbage value.

TOPIC 4 OF 4 • DECLARATION

Declaration: tell the compiler the function exists



Like the **menu board** on the canteen wall: it tells you **Chai, Rs 10** exists before you order. It doesn't show how chai is made.

```
int item_total (int qty, int price) ;
```

return type
what it gives back

name
what we call it

parameters
what it needs

semicolon
ends the declaration

A declaration is also called a **prototype**. It has no body, just a semicolon.

TOPIC 4 OF 4 • DECLARATION

Two ways to arrange a program

A • Define before main

```
#include <stdio.h>

int item_total(int qty, int price) {
    return qty * price;
}

int main(void) {
    printf("%d", item_total(2, 10));
    return 0;
}
```

No prototype needed: the compiler has already read the function.

B • Prototype at the top, define after main

```
#include <stdio.h>

int item_total(int qty, int price);

int main(void) {
    printf("%d", item_total(2, 10));
    return 0;
}

int item_total(int qty, int price) {
    return qty * price;
}
```

main comes first and reads like a story. **Our style.**

PUTTING IT ALL TOGETHER

Our use case: main tells the story, functions do the jobs

`main()`

reads like a story

1. Read the order
2. item_total × 3, add them up
3. apply_discount(total)
4. print_line()
5. Print total and pay, print_line()

`item_total(qty, price)`

`in: 2, 10 → out: 20`

called 3 times

`apply_discount(total)`

`in: 250 → out: 225`

called once

`print_line()`

`in: nothing → out: nothing`

called twice

THE PROGRAM, PART 1 OF 2

main reads like a story

3–5	**2 • Prototypes: the menu board**
13–15	**4 • Three calls, answers added**
16	**4 • Call and keep the answer**
18, 21	**4 • A void call: no answer back**

```
1  #include <stdio.h>
2
3  int item_total(int qty, int price);
4  int apply_discount(int total);
5  void print_line(void);
6
7  int main(void) {
8      int chai, samosa, maggi, total, pay;
9
10     printf("How many chai, samosa, Maggi? ");
11     scanf("%d %d %d", &chai, &samosa, &maggi);
12
13     total = item_total(chai, 10)
14           + item_total(samosa, 15)
15           + item_total(maggi, 40);
16     pay = apply_discount(total);
17
18     print_line();
19     printf("Total: Rs %d\n", total);
20     printf("To pay: Rs %d\n", pay);
21     print_line();
22     return 0;
23 }
```

THE PROGRAM, PART 2 OF 2

The three helpers

25–27 **3 • item_total: in → out**

29–33 **3 • apply_discount: two returns**

35–40 **3 • print_line: void, no return value**

Each one follows the function recipe.

```
25  int item_total(int qty, int price) {
26      return qty * price;
27  }
28
29  int apply_discount(int total) {
30      if (total > 200)
31          return total - total / 10;
32      return total;
33  }
34
35  void print_line(void) {
36      int i;
37      for (i = 0; i < 28; i++)
38          printf("-");
39      printf("\n");
40  }
```

RUN IT YOURSELF

Run the bill, then we'll check your predictions

bill_fn.c

1. Open the link or scan the QR code
2. Type the input: **10 10 0**
3. Press **Run**
4. Check: **Total: Rs 250, To pay: Rs 225**



onlinegdb.com/21VyCsqft

PREDICT, THEN RUN TO CHECK

What will the bill do?

1	Input: 2 1 1. Total? To pay?	Rs 75 and Rs 75
2	Input: 0 0 6 (six Maggi). To pay?	Rs 216: 240 minus 24
3	How many times does item_total run? print_line?	3 times and 2 times

TOPIC 3 OF 4 • CALL AND RETURN • FIX THE BROKEN CODE

Three bugs: the total should be Rs 35

```c
/* Find and fix 3 bugs */
#include <stdio.h>

int item_total(int qty, int price) {
    int t = qty * price;
}

int main(void) {
    int total = 0;
    total = item_total(2);
    item_total(1, 15);
    printf("Total: Rs %d\n", total);
    return 0;
}
```

It should add 2 chai and 1 samosa: **Total: Rs 35**. Read the compiler's messages first.

Lines 5–6: no return t;

Function gives back garbage

Line 10: item_total(2) is missing 10

Compile error: too few arguments

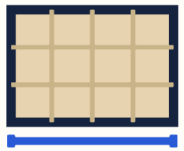
Line 11: the answer is thrown away

Use total = total + item_total(1, 15);

gcc points at two bugs: **“too few arguments”** and **“control reaches end of non-void function”**. The third is silent: only testing finds it.

YOUR TURN • TASK 1 OF 3

Area of a classroom floor



Write a function that returns the area of a rectangle. main reads the length and breadth and prints the area.

The function recipe

1	**Name**	`area`
2	**Inputs**	`int length, int breadth`
3	**Output**	`int`
4	**Prototype**	`int area(int length, int breadth);`
5	**Body**	`return length × breadth`
6	**Test**	`area(8, 5) → 40`

Sample runs

```
Length and breadth? 8 5
Area = 40
```

Check yourself

- ✓ Prototype ends with ;
- ✓ main prints area(l, b) directly
- ✓ Try 0 as one side

Now: Follow the recipe in OnlineGDB: prototype at the top, definition after main.

SOLUTION · THE COMPLETE PROGRAM

Task 1 solution: area()

```c
#include <stdio.h>

int area(int length, int breadth);

int main(void) {
    int l, b;
    printf("Length and breadth? ");
    scanf("%d %d", &l, &b);
    printf("Area = %d\n", area(l, b));
    return 0;
}

int area(int length, int breadth) {
    return length * breadth;
}
```

OUTPUT ION ON (prototype)

[illegible]

main()

l	8
b	5

area()

length	8
breadth	5

▲ returns 40 to main

Output

```
Length and breadth? 8 5
Area = 40
```

YOUR TURN • TASK 2 OF 3

Odd or even seat?



Write is_even(n) that returns 1 if n is even and 0 if odd. main prints which it is.

The function recipe

1	**Name**	`is_even`
2	**Inputs**	`int n`
3	**Output**	`int: 1 or 0`
4	**Prototype**	`int is_even(int n);`
5	**Body**	`return n % 2 == 0;`
6	**Test**	`is_even(7) → 0`

Sample runs

```
Number? 7
7 is odd

Number? 12
12 is even
```

Check yourself

- ✓ A comparison already gives 1 or 0
- ✓ Use it in if (is_even(n))
- ✓ Test 0: it is even

Now: Follow the recipe in OnlineGDB: prototype at the top, definition after main.

SOLUTION · THE COMPLETE PROGRAM

Task 2 solution: is_even()

```c
#include <stdio.h>

int is_even(int n);

int main(void) {
    int n;
    printf("Number? ");
    scanf("%d", &n);
    if (is_even(n))
        printf("%d is even\n", n);
    else
        printf("%d is odd\n", n);
    return 0;
}

int is_even(int n) {
    return n % 2 == 0;
}
```

OUTPUT ION ON (prototype)

[illegible] with a semicolon.

main()

n	7

is_even()

n	7

▲ returns 0 to main

Output

```
Number? 7
7 is odd
```

YOUR TURN • TASK 3 OF 3

Largest of three, as a function



Your Module 1 flowchart, now as a function: largest(a, b, c) returns the biggest number.

The function recipe

1	**Name**	`largest`
2	**Inputs**	`int a, int b, int c`
3	**Output**	`int`
4	**Prototype**	`int largest(int a, int b, int c);`
5	**Body**	`big = a; compare b, then c`
6	**Test**	`largest(4, 4, 4) → 4`

Sample runs

```
Three numbers? 3 9 5
Largest = 9

Three numbers? 4 4 4
Largest = 4
```

Check yourself

- ✓ Same steps as your Module 1 algorithm
- ✓ Test three equal numbers
- ✓ Test the largest in each position

Now: Follow the recipe in OnlineGDB: prototype at the top, definition after main.

SOLUTION · THE COMPLETE PROGRAM

Task 3 solution: largest()

```c
#include <stdio.h>

int largest(int a, int b, int c);

int main(void) {
    int a, b, c;
    printf("Three numbers? ");
    scanf("%d %d %d", &a, &b, &c);
    printf("Largest = %d\n", largest(a, b, c));
    return 0;
}

int largest(int a, int b, int c) {
    int big = a;
    if (b > big) big = b;
    if (c > big) big = c;
    return big;
}
```

OUTPUT ION ON (prototype)

[illegible] with a semicolon.

main()

a	3
b	9
c	5

largest()

a	3
b	9
c	5
big	9

▲ returns 9 to main

Output

```
Three numbers? 3 9 5
Largest = 9
```

TODAY IN ONE SLIDE

Build with blocks

1

Functions

Named blocks that do one job: inputs in, one answer out. Modular programming = building from these blocks.

2

Definition

Ask four questions: job, inputs, output, steps. Then type the header, the body and the return.

3

Call and return

main pauses, a new box opens for the call, return sends one value back, the box closes.

4

Declaration

The prototype, the menu board: name, inputs and output, ending with ;

And the recipe: name, inputs, output, steps, test, then the prototype.

NEXT CLASS

add1(a) changed x to 6, but a stayed 5. Why?

Class 2: call by value and by reference, the stack in the debugger, scope and storage classes.

Before next class: finish one task using the recipe.